

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – SAST SIMULATION

Course Code:	CSA5803	Course Title:	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO3 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1
Weightage:	30% of CIA	Total Marks:	100
CO3 Statement:	Simulate Static Application Security Testing (SAST) using synthetic data		
Student Name:	A Austin	Reg. No.:	192421004
Section / Batch:	2024 B Tech IT	Date:	13/08/26

Instructions to Students

- Ensure all diagrams and workflow illustrations are **original, clear, and professionally formatted**.
- Include **all editable source files** (such as .yaml, .py, requirements.txt, .pre-commit-config.yaml, and any diagram source files if applicable).
- Submit **both** the **GitHub repository link** and the **final PDF report** through **Google Classroom** before the specified deadline.
- Verify that the GitHub repository is accessible and contains all required project files, screenshots, and documentation.

Question Type	Focus Area	Questions	Marks
SAST SIMULATION	Creative Things and concept grasping	Q1	Q1 –100
GRAND TOTAL:		100 Marks	

Code of Conduct

I, A Austin 192421004 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: A Austin

SAST SIMULATION – ASSESSMENT TOOL 1

CSA5803 – DevSecOps Engineering and Application Security

Q1. SAST Simulation

1. Aim

To simulate Static Application Security Testing (SAST) on a small application using synthetic source code, identify common security weaknesses without executing the application, classify the findings by severity, and recommend appropriate remediation.

2. Objective

- Understand the purpose and workflow of SAST in a DevSecOps pipeline.
- Use synthetic source code containing intentionally introduced security weaknesses.
- Detect vulnerabilities by static inspection/scanning before runtime execution.
- Classify findings according to their security impact and explain the risk.
- Document corrective actions and verify that the revised code is safer.

3. SAST Workflow

The simulation follows this sequence: Prepare synthetic code → Define security rules → Perform static scan → Collect findings → Classify severity → Identify root cause → Apply remediation → Re-scan → Record final results.

4. Synthetic Vulnerable Code

```
# Synthetic example for SAST simulation
import sqlite3
```

```
username = input("Enter username: ")
password = input("Enter password: ")
```

```
conn = sqlite3.connect("app.db")
query = "SELECT * FROM users WHERE username='" + username + "' AND password='" + password + "'"
result = conn.execute(query)
```

```
api_key = "TEST-SECRET-12345"
```

```
print("Login result:", result.fetchall())
```

5. Simulated SAST Findings

ID	Finding	Severity	Risk	Recommended Fix
SAST-01	SQL query is constructed using direct user input.	High	An attacker may manipulate the query and access or alter database information.	Use parameterized/prepared SQL statements instead of string concatenation.
SAST-02	Password is included directly in the SQL query.	High	Sensitive authentication data may be exposed or mishandled.	Use secure password verification with properly protected password hashes.
SAST-03	A secret-like API key is hard-coded in source code.	Medium	Secrets stored in source code can be exposed through repositories or logs.	Move secrets to protected environment variables or a secrets manager.
SAST-04	User-controlled input is accepted without validation.	Medium	Unexpected or malicious input can reach application logic.	Validate input according to expected type, length and format.

6. Remediated Code

```
import os
import sqlite3

username = input("Enter username: ").strip()

conn = sqlite3.connect("app.db")
query = "SELECT username, password_hash FROM users WHERE username = ?"
result = conn.execute(query, (username,))

api_key = os.environ.get("APP_API_KEY")

print("User lookup completed.")
```

7. Before-and-After Result

Area	Before SAST	Remediation	After Re-scan
SQL construction	Unsafe string	Parameterized query	Resolved

	concatenation		
Credential handling	Password placed in query	Secure verification approach	Resolved
Secret management	Hard-coded key	Environment variable	Resolved
Input handling	No explicit validation	Trim/validate expected input	Improved

8. DevSecOps Integration

SAST can be integrated into the CI/CD pipeline so that source code is scanned automatically during development. A practical workflow is to run the scanner on every pull request and build, fail the pipeline for agreed critical/high-risk findings, review the reports, remediate vulnerabilities, and repeat the scan before deployment.

9. Key Observations

- SAST analyzes source code without requiring the application to be running.
- Early detection reduces the cost and effort of fixing security defects later in the lifecycle.
- Synthetic vulnerable code is useful for safely demonstrating security scanning concepts.
- Findings should be prioritized according to severity, exploitability, and application impact.
- A re-scan is necessary to confirm that remediation has addressed the identified weakness.

10. Result

The SAST simulation was successfully demonstrated using synthetic source code. The analysis identified insecure SQL construction, unsafe handling of authentication data, hard-coded secret material, and insufficient input validation. After applying secure coding practices and performing a conceptual re-scan, the identified weaknesses were resolved or improved. Thus, the simulation demonstrates how SAST supports early vulnerability detection within a DevSecOps workflow.

11. Conclusion

Static Application Security Testing provides an effective shift-left security mechanism by detecting weaknesses in source code before deployment. Combining automated SAST scans with developer review and remediation helps establish continuous security checks throughout the software development lifecycle.